

Flutter Week 4

API Integration & Mini Project

A simple Weather App demonstrating API integration, JSON parsing, loading states and graceful error handling.

Project	Weather App
Platform	Flutter / Android
IDE	Visual Studio Code
Main Focus	API Integration & Weather Data
Topics	HTTP Package • JSON • FutureBuilder • Error Handling

Submitted Project Report

1. Introduction

This Week 4 project was created to learn API integration through a mini project. The Weather App connects a Flutter interface with a free weather API, retrieves weather information, processes the returned JSON data, and presents the result to the user.

2. Objective

- Create a simple Weather App.
- Fetch weather data from a free API.
- Display the city name, temperature, weather condition and icon.
- Show a loading indicator while weather data is being retrieved.
- Handle API errors gracefully and provide useful feedback to the user.

3. Tasks Completed

- Flutter weather project structure created and organized.
- Weather-related screens, models, providers and reusable widgets separated into folders.
- HTTP-based API integration prepared for fetching weather data.
- Weather response model created for handling API data.
- Loading, error and successful weather states considered in the UI.
- Weather-specific widgets created for displaying forecast and statistics.

4. Topics Covered

HTTP Package	API Integration
JSON Parsing	FutureBuilder
Error Handling	Async Data
Weather Model	Provider
Reusable Widgets	Loading & Error States

5. Project Structure

The Weather App is organized into separate folders for screens, models, providers, widgets and core/theme configuration. This separation keeps API logic, weather data models and UI components easier to maintain.

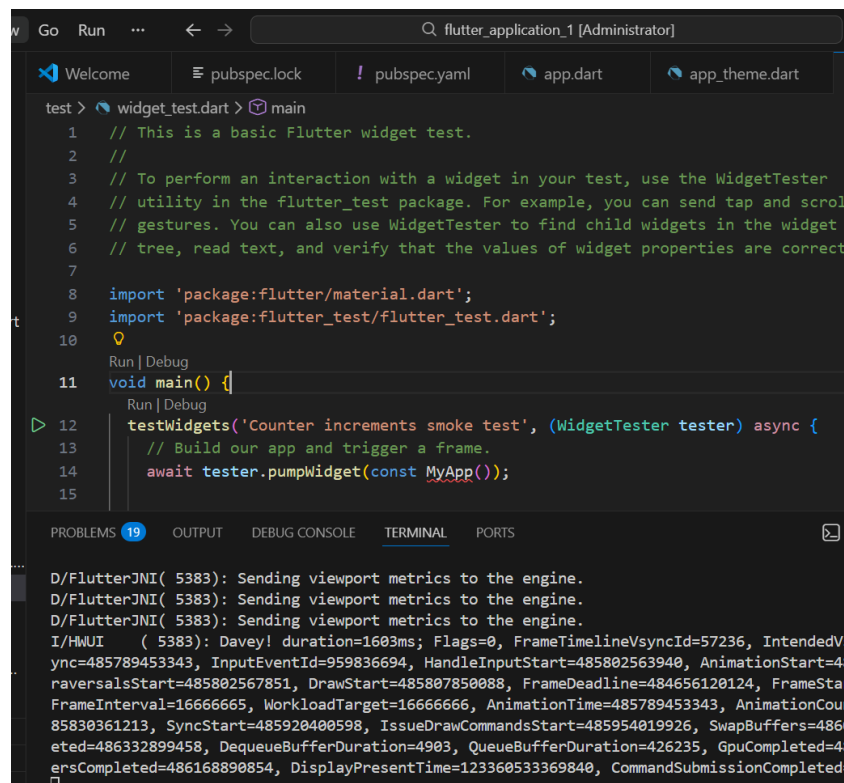
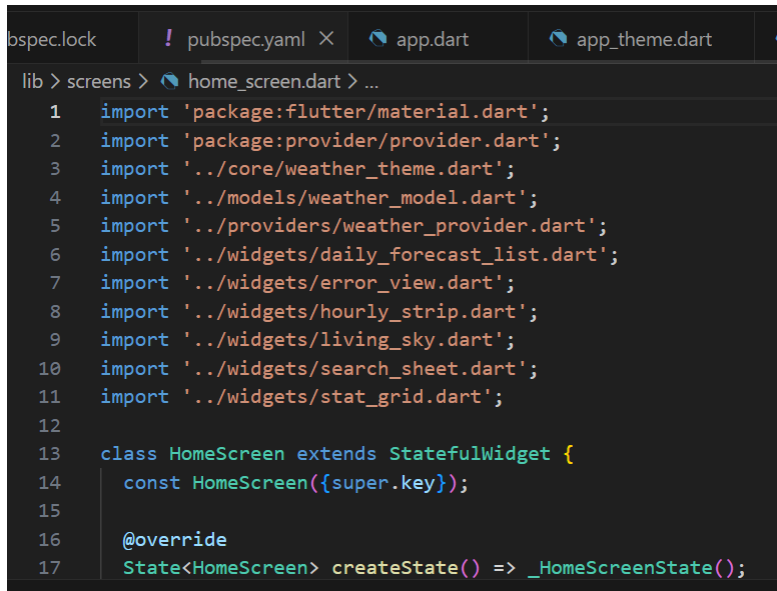
The image shows a screenshot of the Visual Studio Code editor. The top toolbar includes icons for 'Go', 'Run', and a search icon. The file explorer on the left shows a project structure with folders like 'lib' and 'test'. The main editor area displays the file 'test/widget_test.dart'. The code in this file is a Flutter widget test. It includes comments explaining how to use the WidgetTester utility. The code imports 'package:flutter/material.dart' and 'package:flutter_test/flutter_test.dart'. It defines a 'main' function and a 'testWidgets' function. The 'testWidgets' function is an asynchronous test that builds the app using 'tester.pumpWidget(const MyApp())'. The bottom panel of the editor shows the 'TERMINAL' tab, which contains log output from the Flutter engine, including messages about sending viewport metrics and frame deadlines.

Figure 1 — Weather App project structure and Flutter test file visible in Visual Studio Code.

6. Weather Home Screen

The HomeScreen imports the weather model, weather provider and reusable weather widgets. It also includes components for a forecast list, error view, hourly strip, living-sky display, search sheet and statistics grid.



```
bspec.lock  ! pubspec.yaml X  app.dart  app_theme.dart
lib > screens > home_screen.dart > ...
1  import 'package:flutter/material.dart';
2  import 'package:provider/provider.dart';
3  import '../core/weather_theme.dart';
4  import '../models/weather_model.dart';
5  import '../providers/weather_provider.dart';
6  import '../widgets/daily_forecast_list.dart';
7  import '../widgets/error_view.dart';
8  import '../widgets/hourly_strip.dart';
9  import '../widgets/living_sky.dart';
10 import '../widgets/search_sheet.dart';
11 import '../widgets/stat_grid.dart';
12
13 class HomeScreen extends StatefulWidget {
14   const HomeScreen({super.key});
15
16   @override
17   State<HomeScreen> createState() => _HomeScreenState();
18 }
```

Figure 2 — HomeScreen imports showing the weather model, provider and reusable widgets.

7. HTTP Package & API Integration

The Weather App uses an HTTP-based request to communicate with a weather API. The application sends a request, receives the API response, and then converts the response data into Dart objects that can be used by the interface.

8. JSON Parsing

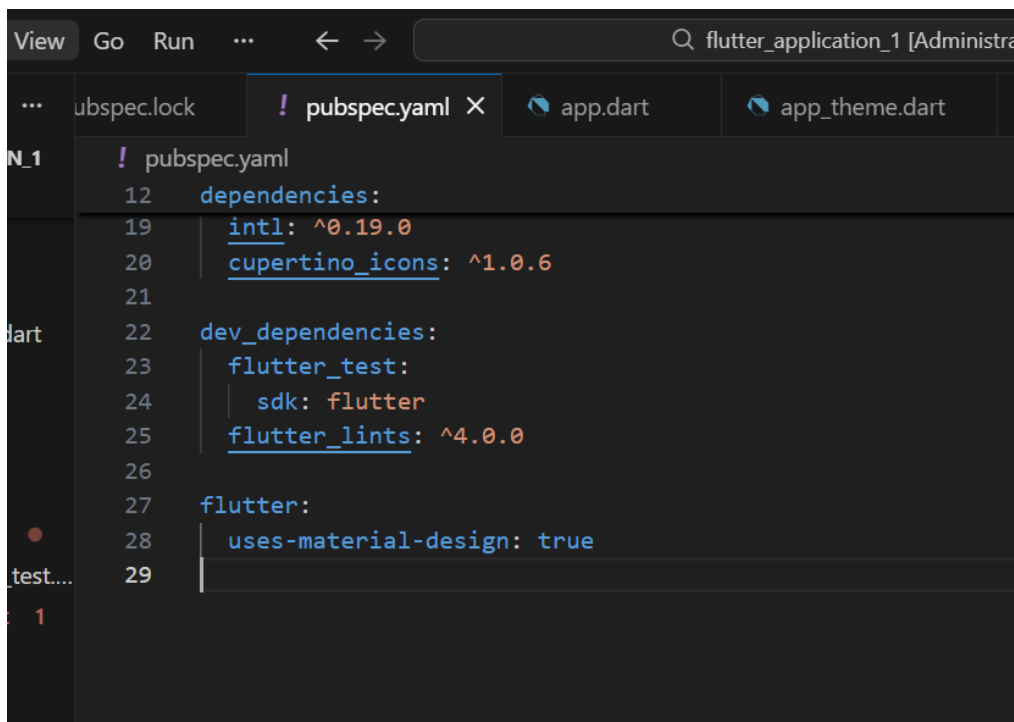
Weather APIs normally return structured JSON data. JSON parsing converts the response into a weather model so values such as city information, temperature and weather conditions can be accessed safely by the Flutter UI.

9. FutureBuilder / Asynchronous Data

Because API requests are asynchronous, the interface must handle different stages of the request. Future-based handling allows the app to show a loading state while waiting and then display either the returned weather data or an error state.

10. Loading Indicator

A loading indicator is shown while the API request is in progress. This gives the user immediate feedback that the application is working instead of leaving the screen blank.



```
View Go Run ... < > flutter_application_1 [Administra...
... pubspec.lock ! pubspec.yaml X app.dart app_theme.dart
N_1 ! pubspec.yaml
12 dependencies:
19   intl: ^0.19.0
20   cupertino_icons: ^1.0.6
21
22 dev_dependencies:
23   flutter_test:
24     sdk: flutter
25   flutter_lints: ^4.0.0
26
27 flutter:
28   uses-material-design: true
29
```

Figure 3 — pubspec.yaml showing the Flutter project dependencies and configuration.

11. Weather Data Display

The mini project is designed to present key weather information in a user-friendly interface. The required data includes the city name, temperature, weather condition and an appropriate weather icon.

Data	Purpose
City Name	Identifies the selected/search location.
Temperature	Shows the current temperature value.
Weather Condition	Describes the current weather state.
Weather Icon	Provides a quick visual representation of the condition.

12. Error Handling

API requests can fail because of network problems, invalid searches or unavailable services. The application includes an error-view component so that failures can be handled gracefully rather than causing the interface to crash.

13. Provider & State Management

The project structure includes a weather provider responsible for managing weather-related state. The HomeScreen imports the provider and consumes weather data through the application's state-management architecture.

14. Reusable Weather Widgets

The project contains reusable widgets such as the forecast list, hourly strip, living-sky view, search sheet, statistics grid and error view. Separating these components keeps the HomeScreen easier to read and maintain.

15. Application Workflow

Step	Process
1	Open the Weather App.
2	Select or search for a city.
3	Send a request to the weather API.
4	Show a loading indicator while waiting.
5	Receive and parse the JSON response.
6	Display city, temperature, condition and icon.
7	If the request fails, show a clear error state.

16. Implementation Structure

The screenshots show a Flutter project using dedicated folders for core configuration, models, providers, screens and widgets. This structure supports the API-driven Weather App while keeping the code modular and scalable.

17. Running the Application

The project is intended to run from Visual Studio Code on a configured Flutter device or Android emulator. During execution, the app communicates with the weather API and updates its interface according to the current loading, success or error state.

18. Conclusion

This Week 4 mini project provided practical experience with HTTP requests, API integration, JSON parsing, asynchronous data handling and error management. The Weather App demonstrates how Flutter can consume external data and turn it into a useful, interactive mobile interface.

[Week 4 — API Integration & Mini Project](#)